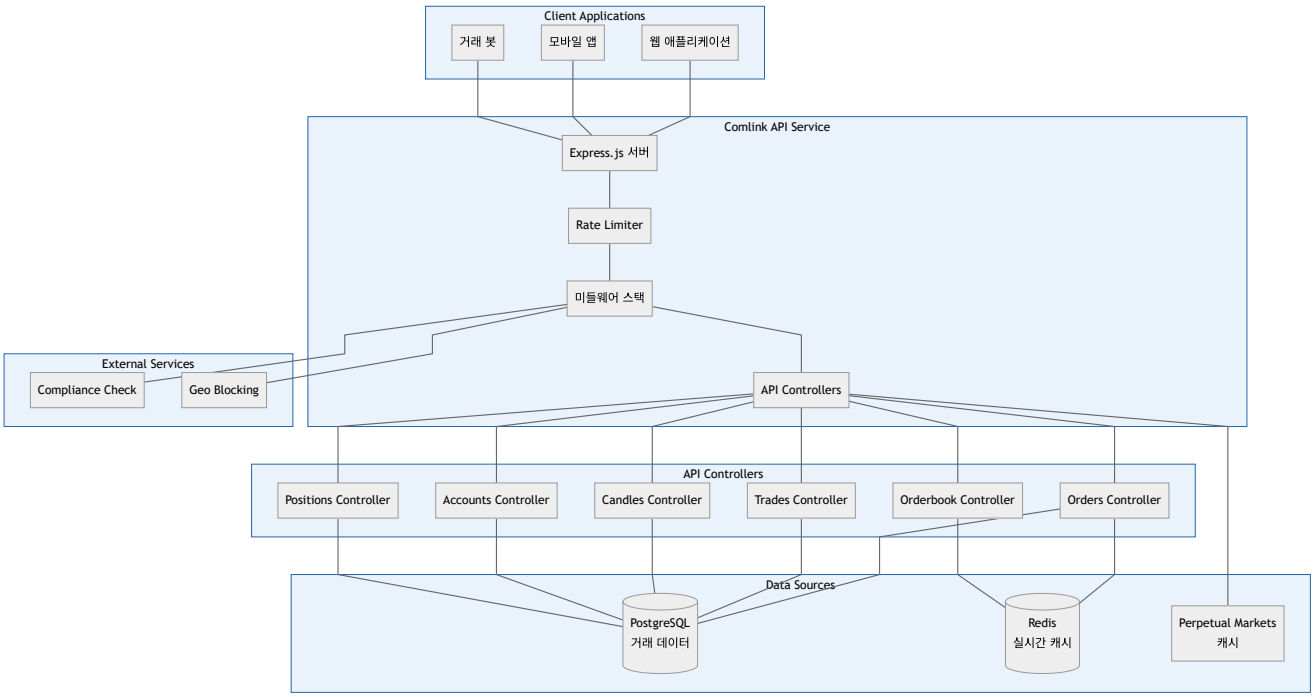
Comlink HTTP API 서비스 구현 분석

개요

Comlink는 dYdX v4 Indexer의 HTTP API 서비스로, 모든 외부 API 요청을 처리하는 중앙 집중식 서비스입니다. Express.js 기반으로 구축되어 있으며, PostgreSQL 데이터베이스와 Redis 캐시를 활용하여 다양한 거래 데이터, 계정 정보, 시장 데이터를 제공합니다.

아키텍처 개요



Express.js 기반 REST API 아키텍처

1. 서버 초기화 및 설정

```
function startServer() {
const app = Server(IndexV4);
const port: number = config.PORT;
const server = app.listen(port, () => {
logger.info({
message: `Api server is listening on ${port}`,
});
});

server.keepAliveTimeout = config.KEEP_ALIVE_MS;
server.headersTimeout = config.HEADERS_TIMEOUT_MS;
}

async function start() {
startBugsnag();

// 영구 선물 마켓 캐시 초기화
await Promise.all([
perpetualMarketRefresher.updatePerpetualMarkets(),
liquidityTierRefresher.updateLiquidityTiers(),
]);
```

```
// 백그라운드에서 캐시 업데이트
wrapBackgroundTask(perpetualMarketRefresher.start(), true, 'startUpdatePerpetual
wrapBackgroundTask(liquidityTierRefresher.start(), true, 'startUpdateLiquidityTi

await connectToRedis();
startServer();
}
```

2. 미들웨어 스택 구성

```
export default function server(indexV4?: express.Router): Express {
const app: Express = express();

// 응답 시간 측정
app.use(responseTime({ suffix: false }));

// 요청 ID 생성
app.use(requestId());

// 응답 본문 캡처 (로깅용)
app.use(resBodyCapture);

// CORS 설정
```

```
const corsOptions = {
origin: config.CORS_ORIGIN,
optionsSuccessStatus: 200,
};
app.use(cors(corsOptions));

// 캐시 비활성화
app.use(nocache());

// 헬스 체크 엔드포인트
app.get('/health', (_req: express.Request, res: express.Response) => {
res.json({ ok: true });
});

// JSON 파싱
app.use(bodyParser.json());

// 요청 로깅
app.use(RequestLogger);

// API 라우터 등록
if (indexV4) {
app.use('/v4', indexV4);
```

```
}

// Swagger 문서
app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerJson));

// 에러 핸들링
app.use(logErrors);

// 404 처리
app.use((_req: express.Request, res: express.Response) => {
res.status(404).json({
error: 'Not Found',
errorCode: 404,
});
});

return app;
}
```

3. API 라우터 구조

```
const router: express.Router = express.Router();
```

```
// 알파벳 순서로 라우터 등록

router.use('/addresses', AddressesController);

router.use('/assetPositions', AssetPositionsController);

router.use('/candles', CandlesController);

router.use('/fills', FillsController);

router.use('/height', HeightController);

router.use('/historicalBlockTradingRewards', HistoricalBlockTradingRewardControl

router.use('/historicalFunding', HistoricalFundingController);

router.use('/historical-pnl', PnlticksController);

router.use('/historicalTradingRewardAggregations', HistoricalTradingRewardContro

router.use('/orders', OrdersController);

router.use('/orderbooks', OrderbooksController);

router.use('/perpetualMarkets', PerpetualMarketController);

router.use('/perpetualPositions', PerpetualPositionsController);

router.use('/sparklines', SparklinesController);

router.use('/time', TimeController);

router.use('/trades', TradesController);

router.use('/transfers', TransfersController);

router.use('/screen', ComplianceController);

router.use('/compliance', ComplianceV2Controller);
```

주요 API 엔드포인트별 구현 방식

1. Orders Controller (주문 관리)

주문 목록 조회 (`GET /orders`)

```
@Route('orders')
class OrdersController extends Controller {
@Get('/')
async listOrders(
@Query() address: string,
@Query() subaccountNumber: number,
@Query() limit?: number,
@Query() ticker?: string,
@Query() side?: OrderSide,
@Query() type?: OrderType,
@Query() status?: APIOrderStatus[],
@Query() goodTilBlockBeforeOrAt?: number,
@Query() goodTilBlockTimeBeforeOrAt?: IsoString,
@Query() returnLatestOrders?: boolean,
): Promise<OrderResponseObject[]> {
// CLOB Pair ID 조회
let clobPairId: string | undefined;
if (ticker !== undefined) {
clobPairId = perpetualMarketRefresher.getClobPairIdFromTicker(ticker);
}
```

```
// 쿼리 설정 구성
const  orderQueryConfig: OrderQueryConfig = {
subaccountId: [SubaccountTable.uuid(address, subaccountNumber)],
limit,
clobPairId,
side,
type,
goodTilBlockBeforeOrAt:  goodTilBlockBeforeOrAt?.toString(),
goodTilBlockTimeBeforeOrAt,
};


// PostgreSQL과 Redis에서 병렬로 데이터 조회
const [redisOrderMap, postgresOrders] = await  Promise.all([
getRedisOrderMapForSubaccountId(
SubaccountTable.uuid(address, subaccountNumber),
clobPairId,
side,
type,
goodTilBlockBeforeOrAt,
goodTilBlockTimeBeforeOrAt,
),
OrderTable.findAll(orderQueryConfig, [], {
```

```
...DEFAULT_POSTGRES_OPTIONS,

orderBy: [

[OrderColumns.goodTilBlock, ordering],

[OrderColumns.goodTilBlockTime, ordering],

],

}),

]);



// Redis에만 있는 주문들을 PostgreSQL에서 추가 조회

const redisOrderIds: string[] = _.map(

Object.values(redisOrderMap),

(redisOrder: RedisOrder) => OrderTable.orderIdToUuid(redisOrder.order!.orderId!

);

const postgresOrderIdsToFetch: string[] = _.difference(

redisOrderIds,

_.map(postgresOrders, OrderColumns.id),

);



let additionalPostgresOrders: OrderFromDatabase[] = [];

if (!_.isEmpty(postgresOrderIdsToFetch)) {

additionalPostgresOrders = await OrderTable.findAll({

id: redisOrderIds,

}, [], { ...DEFAULT_POSTGRES_OPTIONS });
```

```
}

// PostgreSQL과 Redis 데이터 병합
const postgresOrderMap: PostgresOrderMap = _.keyBy(
_.concat(postgresOrders, additionalPostgresOrders),
OrderColumns.id,
);

let mergedResponses: OrderResponseObject[] = mergePostgresAndRedisOrdersToRespo
postgresOrderMap,
redisOrderMap,
);

// 상태 필터링
if (status !== undefined) {
mergedResponses = _.filter(mergedResponses, (orderResponse: OrderResponseObject)
return status.includes(orderResponse.status);
});
}

return sortAndLimitResponses(mergedResponses, ordering, limit || config.API_LIM
}
```

```
}
```

Redis 주문 데이터 조회 최적화

```
async function getRedisOrderMapForSubaccountId(
subaccountId: string,
clobPairId?: string,
side?: OrderSide,
type?: OrderType,
goodTilBlockBeforeOrAt?: number,
goodTilBlockTimeBeforeOrAt?: IsoString,
): Promise<RedisOrderMap> {
// LIMIT 주문만 Redis에서 지원
if (type !== undefined && type !== OrderType.LIMIT) {
return {};
}

// 서브계정의 모든 주문 ID 조회
const subaccountOrderIds: string[] = await SubaccountOrderIdsCache.getOrderIds
subaccountId,
redisClient,
);
```

```
// 병렬로 주문 데이터 조회
const nullableRedisOrders: (RedisOrder | null)[] = await Promise.all(
_.map(subaccountOrderIds, (orderId: string) => OrdersCache.getOrder(orderId, re
);

// 필터링 로직
const redisOrders: RedisOrder[] = _.filter(nullableRedisOrders, (redisOrder: Re
if (redisOrder === null) return false;

// CLOB Pair ID 필터링
const redisClobPairId: string = redisOrder!.order!.orderId!.clobPairId.toString
if (clobPairId !== undefined && redisClobPairId !== clobPairId) {
return false;
}

// 주문 사이드 필터링
const redisSide: OrderSide = protocolTranslations.protocolOrderSideToOrderSide(
redisOrder!.order!.side
);
if (side !== undefined && side !== redisSide) {
return false;
}
```

```
// goodTilBlock 필터링

const redisGoodTilBlock: number | undefined = protocolTranslations

.getGoodTilBlock(redisOrder!.order!);

if (redisGoodTilBlock !== undefined) {

if (goodTilBlockBeforeOrAt !== undefined && redisGoodTilBlock > goodTilBlockBefo

return false;

}

} else if (goodTilBlockBeforeOrAt !== undefined) {

return false;

}



// goodTilBlockTime 필터링

const redisGoodTilBlockTime: string | undefined = protocolTranslations

.getGoodTilBlockTime(redisOrder!.order!);

if (redisGoodTilBlockTime) {

const redisGoodTilBlockTimeDateObj: DateTime = DateTime.fromISO(redisGoodTilBlo

if (goodTilBlockTimeBeforeOrAt !== undefined &&

redisGoodTilBlockTimeDateObj > DateTime.fromISO(goodTilBlockTimeBeforeOrAt)) {

return false;

}

} else if (goodTilBlockTimeBeforeOrAt !== undefined) {

return false;

}
```

```
return true;
}) as RedisOrder[];

return _.keyBy(redisOrders, 'id');
}
```

2. Orderbook Controller (주문서 관리)

```
@Route('orderbooks')
class OrderbookController extends Controller {
@Get('/perpetualMarket/:ticker')
async getPerpetualMarket(@Path() ticker: string): Promise<OrderbookResponseObje
// 영구 선물 마켓 정보 조회
const perpetualMarket: PerpetualMarketFromDatabase | undefined = perpetualMarke
.getPerpetualMarketFromTicker(ticker);

if (perpetualMarket === undefined) {
throw new NotFoundError(`${ticker} not found in markets of type ${MarketType.P
}

// Redis에서 주문서 레벨 조회
const orderbookLevels: OrderbookLevels = await OrderbookLevelsCache.getOrderBo
```

```
ticker,
redisClient,
{
sortSides: true, // 가격 순 정렬
uncrossBook: true, // 교차된 주문서 정리
limitPerSide: config.API_ORDERBOOK_LEVELS_PER_SIDE_LIMIT, // 사이드별 제한
},
);

return OrderbookLevelsToResponseObject(orderbookLevels, perpetualMarket);
}
}
```

3. Trades Controller (거래 내역)

```
@Route('trades')
class TradesController extends Controller {
@Get('/perpetualMarket/:ticker')
async getTrades(
@Path() ticker: string,
@Query() limit?: number,
@Query() createdBeforeOrAtHeight?: number,
```

```
@Query() createdBeforeOrAt?: IsoString,

): Promise<TradeResponse> {

// 티커에서 CLOB Pair ID 조회

const  clobPairId: string | undefined = perpetualMarketRefresher

.getClobPairIdFromTicker(ticker);



if (clobPairId === undefined) {

throw  new  NotFoundError(`${ticker} not found in tickers of type ${MarketType.P

}



// PostgreSQL에서 체결 데이터 조회 (TAKER만)

const  fills: FillFromDatabase[] = await  FillTable.findAll(

{

clobPairId,

liquidity:  Liquidity.TAKER, // 공개 거래는 TAKER만

limit,

createdBeforeOrAtHeight:  createdBeforeOrAtHeight?.toString(),

createdBeforeOrAt,

},

[QueryableField.LIQUIDITY, QueryableField.CLOB_PAIR_ID, QueryableField.LIMIT],

);



return {
```

```
trades: fills.map((fill: FillFromDatabase): TradeResponseObject => {
return fillToTradeResponseObject(fill);
}),
};
}
}
```

4. 데이터 변환 및 응답 객체 생성

PostgreSQL과 Redis 데이터 병합

```
export function mergePostgresAndRedisOrdersToResponseObjects(
postgresOrderMap: PostgresOrderMap,
redisOrderMap: RedisOrderMap,
): OrderResponseObject[] {
const orderIds: string[] = _.uniq(
Object.keys(redisOrderMap).concat(Object.keys(postgresOrderMap)),
);

return _.map(orderIds, (orderId: string) => {
return postgresAndRedisOrderToResponseObject(
postgresOrderMap[orderId],
redisOrderMap[orderId],
) as OrderResponseObject;
```

```
});
}

export function postgresAndRedisOrderToResponseObject(
postgresOrder: OrderFromDatabase | undefined,
redisOrder?: RedisOrder | null,
): OrderResponseObject | undefined {
// PostgreSQL 데이터가 없으면 Redis 데이터만 사용
if (postgresOrder === undefined) {
if (redisOrder === null || redisOrder === undefined) {
return undefined;
}
return redisOrderToResponseObject(redisOrder);
}

// PostgreSQL 데이터를 기본으로 사용
const orderResponse: OrderResponseObject = postgresOrderToResponseObject(postgr
// Redis 데이터가 있으면 실시간 필드 오버라이드
if (redisOrder === null || redisOrder === undefined) {
return orderResponse;
}
const redisOrderTIF: TimeInForce = protocolTranslations.protocolOrderTIFToTIF(
redisOrder.order!.timeInForce,
```

```
);

return {
...orderResponse,
size: redisOrder.size, // 실시간 주문 크기
price: redisOrder.price, // 실시간 가격
goodTilBlock: protocolTranslations.getGoodTilBlock(redisOrder.order!)?.toString
goodTilBlockTime: protocolTranslations.getGoodTilBlockTime(redisOrder.order!),
timeInForce: apiTranslations.orderTIFToAPITIF(redisOrderTIF),
postOnly: apiTranslations.isOrderTIFPostOnly(redisOrderTIF),
reduceOnly: redisOrder.order!.reduceOnly,
};
}
```

PostgreSQL 쿼리 최적화 및 성능 튜닝

1. 쿼리 최적화 전략

인덱스 활용 최적화

```
// 주문 조회 시 복합 인덱스 활용
const postgresOrders = await OrderTable.findAll(
orderQueryConfig,
[QueryableField.SUBACCOUNT_ID, QueryableField.CLOB_PAIR_ID, QueryableField.SIDE]
```

```
{
...DEFAULT_POSTGRES_OPTIONS,
orderBy: [
[OrderColumns.goodTilBlock, ordering],
[OrderColumns.goodTilBlockTime, ordering],
],
}
);
```

페이지네이션 최적화

```
// 커서 기반 페이지네이션 사용
const fills: FillFromDatabase[] = await FillTable.findAll(
{
clobPairId,
liquidity: Liquidity.TAKER,
limit,
createdBeforeOrAtHeight: createdBeforeOrAtHeight?.toString(), // 높이 기반 커서
createdBeforeOrAt, // 시간 기반 커서
},
[QueryableField.LIQUIDITY, QueryableField.CLOB_PAIR_ID, QueryableField.LIMIT],
);
```

2. 캐싱 전략

영구 선물 마켓 캐시

```
// 백그라운드에서 주기적 캐시 업데이트
await Promise.all([
perpetualMarketRefresher.updatePerpetualMarkets(),
liquidityTierRefresher.updateLiquidityTiers(),
]);

wrapBackgroundTask(perpetualMarketRefresher.start(), true, 'startUpdatePerpetual
wrapBackgroundTask(liquidityTierRefresher.start(), true, 'startUpdateLiquidityTi

// 캐시에서 빠른 조회
const perpetualMarket = perpetualMarketRefresher.getPerpetualMarketFromTicker(t
const clobPairId = perpetualMarketRefresher.getClobPairIdFromTicker(ticker);
```

Redis 캐시 활용

```
// 주문서 레벨 캐시에서 조회
const orderbookLevels: OrderbookLevels = await OrderbookLevelsCache.getOrderBo
ticker,
redisClient,
{
```

```
sortSides: true,
uncrossBook: true,
limitPerSide: config.API_ORDERBOOK_LEVELS_PER_SIDE_LIMIT,
},
);

// 개별 주문 캐시에서 조회
const redisOrder = await OrdersCache.getOrder(orderId, redisClient);
```

3. 성능 모니터링

```
// 각 엔드포인트별 응답 시간 측정
router.get('/perpetualMarket/:ticker',
// ... 미들웨어들
async (req: express.Request, res: express.Response) => {
const start: number = Date.now();
try {
const controller: OrderbookController = new OrderbookController();
return res.send(await controller.getPerpetualMarket(ticker));
} catch (error) {
return handleControllerError('OrderbooksController GET /perpetualMarket/:ticker
} finally {
// 성능 메트릭 수집
```

```
stats.timing(
`${config.SERVICE_NAME}.${controllerName}.get_orderbooks.timing`,
Date.now() - start,
);
}
},
);
```

Rate Limiting 및 보안

1. Rate Limiting 구현

```
export function rateLimiterMiddleware(
rateLimiter: RateLimiterRedis,
postfixKey?: string,
) {
return async (
req: express.Request,
res: express.Response,
next: express.NextFunction,
) => {
if (!config.RATE_LIMIT_ENABLED) {
return next();
```

```
}

const ipAddr: string | undefined = getIpAddr(req);
if (ipAddr === undefined) {
return create4xxResponse(res, 'Too many requests', 429);
}

// 내부 IP는 제한 없음
const pointCost: number = getPointCost(ipAddr);
// Redis 키 생성 (IP + 선택적 postfix)
const postfix: string | undefined = postfixKey ? _.get(req, postfixKey) : undef
const redisKey: string = postfix ? ipAddr.concat(postfix) : ipAddr;

try {
const limitRes: RateLimiterRes = await rateLimiter.consume(redisKey, pointCost
// Rate limit 헤더 설정
res.set({
'RateLimit-Remaining': limitRes.remainingPoints,
'RateLimit-Reset': Date.now() + limitRes.msBeforeNext,
'RateLimit-Limit': rateLimiter.points,
});
} catch (reject) {
if (reject instanceof Error) {
```

```
logger.error({ message: 'redis error when checking rate limit', reject });

} else {

const rejectRes: RateLimiterRes = reject as RateLimiterRes;

res.set({

'RateLimit-Remaining': rejectRes.remainingPoints,

'RateLimit-Reset': Date.now() + rejectRes.msBeforeNext,

'Retry-After': rejectRes.msBeforeNext,

'RateLimit-Limit': rateLimiter.points,

});

return create4xxResponse(res, 'Too many requests', 429);

}

}


return next();

};

}


export function getPointCost(ipAddress: string): number {

if (isIndexerIp(ipAddress)) {

return 0; // 내부 IP는 비용 없음

}

return 1; // 외부 IP는 1 포인트
```

```
}
```

2. 컴플라이언스 및 지역 차단

```
// 컴플라이언스 및 지역 확인 미들웨어
export const complianceAndGeoCheck = async (
req: express.Request,
res: express.Response,
next: express.NextFunction,
): Promise<void> => {
// 특정 엔드포인트에서만 확인
if (!shouldCheckCompliance(req)) {
return next();
}

const ipAddr: string | undefined = getIpAddr(req);
if (ipAddr === undefined) {
return create4xxResponse(res, 'Unable to determine IP address', 400);
}

try {
// 지역 및 컴플라이언스 확인
const complianceData = await checkComplianceAndGeo(ipAddr, req.params.address)
```

```
if (!complianceData.compliant) {
return create4xxResponse(res, 'Access restricted', 403);
}
return next();
} catch (error) {
logger.error({ message: 'Compliance check failed', error });
return create4xxResponse(res, 'Compliance check failed', 500);
}
};
```

3. 입력 검증 및 새니타이제이션

```
// 배열 입력 새니타이제이션
export function sanitizeArray(value: any): any[] {
if (Array.isArray(value)) {
return value;
}
if (typeof value === 'string') {
return value.split(',').map(item => item.trim());
}
return [];
}
```

```
// 배열 유효성 검증

export function validateArray(inputArray: any[], validValues: any[]): boolean

if (!Array.isArray(inputArray)) {

return false;

}

return inputArray.every(item => validValues.includes(item));

}



// 스키마 기반 검증

const CheckSubaccountSchema = [

...checkSchema({

address: {

in: 'query',

isString: true,

isLength: { options: { min: 1 } },

},

subaccountNumber: {

in: 'query',

isInt: { options: { min: 0, max: 127 } },

},

}),

];
```

Swagger/OpenAPI 문서화 시스템

1. TSOA 기반 자동 문서 생성

```
// tsoa.json 설정
{
"entryFile": "src/index.ts",
"noImplicitAdditionalProperties": "throw-on-extras",
"spec": {
"outputDirectory": "public",
"specVersion": 3,
"title": "Indexer API"
},
"swagger": {
"entryFile": "./src/index.ts"
},
"swaggerJson": "swagger.json",
"routes": {
"routesDir": "src"
}
}
```

2. 컨트롤러 어노테이션

```
@Route('orders')
class OrdersController extends Controller {
/**
* 주문 목록을 조회합니다.
* @param address 지갑 주소
* @param subaccountNumber 서브계정 번호
* @param limit 조회할 주문 수 제한
* @param ticker 마켓 티커 (선택사항)
* @param side 주문 사이드 (선택사항)
* @param type 주문 타입 (선택사항)
* @param status 주문 상태 배열 (선택사항)
*/
@Get('/')
async listOrders(
@Query() address: string,
@Query() subaccountNumber: number,
@Query() limit?: number,
@Query() ticker?: string,
@Query() side?: OrderSide,
@Query() type?: OrderType,
@Query() status?: APIOrderStatus[],
@Query() goodTilBlockBeforeOrAt?: number,
```

```
@Query() goodTilBlockTimeBeforeOrAt?: IsoString,

@Query() returnLatestOrders?: boolean,

): Promise<OrderResponseObject[]> {

// 구현...

}

}
```

3. 자동 문서 생성 및 서빙

```
// 빌드 시 Swagger 문서 자동 생성

"scripts": {

"swagger":  "tsoa spec-and-routes",

"build":  "pnpm run swagger && tsc"

}
```

```
// Express에서 Swagger UI 서빙

app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerJson));
```

4. API 문서 예시

생성된 API 문서는 다음과 같은 정보를 포함합니다:

- **엔드포인트 목록**: 모든 사용 가능한 API 엔드포인트
- **요청/응답 스키마**: 각 엔드포인트의 입력 및 출력 형식
- **파라미터 설명**: 쿼리 파라미터, 경로 파라미터, 본문 파라미터

- **에러 코드**: 가능한 HTTP 상태 코드 및 에러 메시지
- **예시 요청**: cURL, JavaScript, Python 등의 코드 샘플

에러 처리 및 로깅

1. 중앙집중식 에러 처리

```
export function handleControllerError(
endpoint: string,
message: string,
error: any,
req: express.Request,
res: express.Response,
): express.Response {
logger.error({
at: endpoint,
message,
error: error.message,
stack: error.stack,
requestId: req.id,
url: req.url,
method: req.method,
});
```

```
if (error instanceof NotFoundError) {
return res.status(404).json({
error: error.message,
errorCode: 404,
});
}

if (error instanceof ValidationError) {
return res.status(400).json({
error: error.message,
errorCode: 400,
});
}

// 기본 서버 에러
return res.status(500).json({
error: 'Internal Server Error',
errorCode: 500,
});
}
```

2. 요청/응답 로깅

```
const RequestLogger = (req: express.Request,
res: express.Response, next: express.NextFunction) => {

const start = Date.now();

// 요청 로깅

logger.info({

at: 'RequestLogger',

message: 'Incoming request',

method: req.method,

url: req.url,

requestId: req.id,

userAgent: req.get('User-Agent'),

ip: getIpAddr(req),

});



// 응답 완료 시 로깅

res.on('finish', () => {

const duration = Date.now() - start;

logger.info({

at: 'RequestLogger',

message: 'Request completed',

method: req.method,

url: req.url,

requestId: req.id,
```

```
statusCode: res.statusCode,
duration,
});
});

next();
};
```

3. 성능 메트릭 수집

```
// 응답 시간 측정 미들웨어
app.use(responseTime({ suffix: false }));

// 컨트롤러별 성능 메트릭
stats.timing(
`${config.SERVICE_NAME}.${controllerName}.${methodName}.timing`,
Date.now() - start,
{ success: success.toString() }
);

// 응답 코드 통계
ExportResponseCodeStats({ controllerName })
```

결론

Comlink 서비스는 dYdX v4의 모든 외부 API 요청을 처리하는 중앙 집중식 HTTP API 서버로, Express.js 기반의 견고한 아키텍처를 제공합니다. PostgreSQL과 Redis를 효율적으로 활용한 데이터 조회 최적화, 포괄적인 Rate Limiting 및 보안 기능, 자동화된 API 문서 생성, 그리고 체계적인 에러 처리 및 모니터링을 통해 높은 성능과 안정성을 달성하고 있습니다.